

A. Concurrency Control (8 points)

Consider the following schedule:

$$S = r_1(X) w_1(Y) r_2(X) w_3(Z) w_5(X) r_5(Z) r_6(Y) r_3(Y) r_3(Z) r_6(X) r_2(Y) w_6(Y) r_6(Z)$$

Establish membership or non-membership of S with respect to the serializability classes VSR, CSR, 2PL, Strict 2PL, TS Mono, TS Multi (with the usual conventions adopted in practice). When considering locks, explicitly indicate lock upgrades/downgrades, if any. Justify your answers with terse but cogent explanations.

Solution.

Strict 2PL

Schedule S is not a member of Strict 2PL, because T_2 needs to release a lock before its commit time, as shown below.

	1	2	3	4	5	6	7	8	9	10	11	12	13	
X	r_1	r_2			$\downarrow_2 w_5$					r_6				
Y		w_1					$r_6 r_3$			r_2	$\downarrow_2^2$	w_6		
Z			w_3		$\downarrow_3 r_5$				r_3	$\downarrow_3^3$			r_6	

Legend

$\downarrow$: mandatory unlock request

$\downarrow$: commit time

2PL

Schedule S belongs to 2PL, as the following table shows.

	1	2	3	4	5	6	7	8	9	10	11	12	13	
X	$\uparrow_1 r_1$	$\uparrow_2 \downarrow_1 r_2$			$\downarrow_2 \uparrow_5 w_5$	$\downarrow_5$			$\uparrow_6 r_6$				$\downarrow_6$	
Y	$\uparrow_1$	w_1	$\downarrow_1 \uparrow_2 \uparrow_3$				$\uparrow_6 r_6 r_3$	$\downarrow_3$	$r_2 \downarrow_2$	$\uparrow_6 w_6$	$\downarrow_6$			
Z			$\uparrow_3 w_3$	$\downarrow_3$	$\uparrow_5 r_5$	$\downarrow_5$	$r_3 \downarrow_3$			$\uparrow_6$	r_6	$\downarrow_6$		

Legend

$\uparrow$: read lock request

$\uparrow$: write lock request

$\uparrow$: lock upgrade

$\downarrow$: lock downgrade

$\downarrow$: unlock request

VSR and CSR

As a consequence of 2PL membership, S also belongs to VSR and CSR.

TS Mono

The schedule is in TS Mono, because no transaction needs to be killed, as shown below.

Operation	RTM(X)	WTM(X)	RTM(Y)	WTM(Y)	RTM(Z)	WTM(Z)	Killed transactions
$r_1(X)$	1 (0)	0	0 (0)	0	0 (0)	0	
$w_1(Y)$				1			
$r_2(X)$	2 (0)						
$w_3(Z)$						3	
$w_5(X)$		5					
$r_5(Z)$					5 (0)		
$r_6(Y)$			6 (0)				
$r_3(Y)$			6 (0)				
$r_3(Z)$					5 (0)		
$r_6(X)$	6 (0)						
$r_2(Y)$			6 (0)				
$w_6(Y)$				6			
$r_6(Z)$					6 (0)		

TS Multi

Clearly, since S belongs to TS Mono, it must also belong to TS Multi.

B. Physical Databases (8 points)

A chain managing swimming pools and waterparks stores data about their customers and sold tickets. Table **CUSTOMER** (**Customer_ID**, **Name**, **Surname**, **City**, **Birth_Date**) contains 80K tuples primarily entry-sequenced on 1K blocks. Table **TICKET** (**Ticket_ID**, **Customer_ID**, **Pool_Name**, **Entrance_Date**, **Type**, **WithDeckChair**, ..., **TotalCost**) stores 800K tuples sequentially-ordered by **Customer_ID** on 4K blocks. Describe the most efficient execution plan you can think of for the following query (and estimate its cost) in the two scenarios below, considering that 1% of the customers have bought tickets for at least 5 different swimming pools and 10% of customers are over 65. Write the formulas of the total costs and explain in detail the execution plans (in particular, where the main clauses of the query are evaluated). Describe in a concise way also the discarded execution plans.

```
SELECT Customer_ID, Name, Surname
FROM CUSTOMER
WHERE Birth_Date < 30/01/1958 AND Customer_ID IN
  (SELECT Customer_ID FROM TICKET
   GROUP BY Customer_ID
   HAVING COUNT(DISTINCT Pool_Name) >= 5);
```

1. The only secondary structure on **CUSTOMER** is a secondary hash with $h(\text{Customer_ID})$, containing 160 buckets and no overflow chains.
2. There is also a B+ index on **TICKET** with 3 levels and 3K leaf nodes, with composite search key (**Customer_ID**, **Entrance_Date**, **Pool_Name**).

Solution.

1. **Strategy 1:** Scan **CUSTOMER** and for each customer over 65 (they are $80K \cdot 10\%$) scan **TICKET**: Once the full tuples of the tickets of the customer have been found, count the number of distinct pools.

Cost 1.1: $1K (\text{scan CUSTOMER}) + 80K \cdot 10\% \cdot 4K (\text{scan Ticket}) = \mathbf{32M!!!}$ I/O accesses

(4K in the worst case, when the **Customer_ID** is at the end of the file - in general, it will be $< 4K$; in the average case 2K (**16M** I/O accesses).)

Strategy 2: Scan **TICKET**, count the tickets by customer and lookup the customer using the hash index.

Only 1% of the customers satisfy the **HAVING** condition, i.e., $80K \cdot 1\% = 800$. On average, they bought $800K/80K = 10$ tickets. One block in the **TICKET** table contains $800K/4K = 200$ tuples. While visiting a block, it is possible to count the number of tickets of the customer and check if the pools are different.

Cost 1.2: $4K (\text{scan TICKET}) + 80K \cdot 1\% \cdot (1 (\text{access to hash index}) + 1 (\text{access to the full tuple})) = \mathbf{5,6K}$ I/O accesses $\rightarrow$ MUCH BETTER!

Strategy 2bis: Scan **TICKET**, count the tickets by customer and cache the 800 **Customer_ID**s satisfying the conditions of the nested query. Then, scan the hash index to search the 800 cached **Customer_ID** values, and follow the 800 pointers that are found.

Cost 1.2bis: $4K (\text{scan TICKET}) + 160 (\text{scan the hash index}) + 80K \cdot 1\% \cdot 1 (\text{access to the full tuple for the 800 customers}) = \mathbf{4,96K}$ I/O accesses $\rightarrow$ EVEN BETTER!

2. Strategy 1 of the previous scenario can be improved: after scanning the customers, their tickets can be reached through the B+. Since there are about 10 tickets for each customer ($800K \text{ tickets} / 80K \text{ customers}$), all the composite keys will be found in the same leaf node (each leaf node contains $800K/3K$

composite keys, which is much greater than the 10 tickets of each customer). Notice that we do not need to access the full **TICKET** tuples in the primary structure to count the number of distinct pools, since we have both the **Customer.ID** and the **Pool.Name** in the leaf nodes.

Cost 2.1: $1K \text{ (scan CUSTOMER)} + 80K * 10\% * 3 \text{ (B+ levels)} = \mathbf{25K}$ I/O accesses.

Also Strategies 2 and 2bis can be improved, by scanning the leaf nodes of the B+ tree instead of the primary structure. The costs become, respectively:

Cost 2.2: $2 \text{ (B+ intermediate levels)} + 3K \text{ (scan B+ leaf nodes)} + 80K * 1\% * (1 \text{ (access to hash index)} + 1 \text{ (access to the full tuple)}) = \mathbf{4,6K}$ I/O accesses.

Cost 2.2bis: $2 \text{ (B+ intermediate levels)} + 3K \text{ (scan B+ leaf nodes)} + 160 \text{ (scan the hash index)} + 80K * 1\% * 1 \text{ (access to the full tuple)} = \mathbf{3,96K}$ I/O accesses.

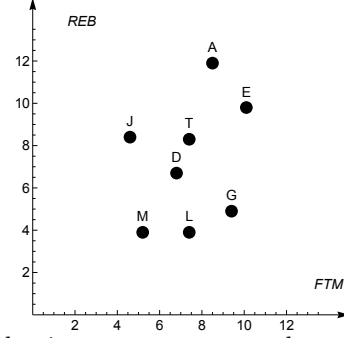
C. Ranking (9 points)

The following dataset D_1 reports NBA stats regarding Free Throws Made (FTM), Rebounds (REB) and Assists (AST) for some of the current best players (higher values are better).

Dataset D_1 :

	<i>FTM</i>	<i>REB</i>	<i>AST</i>
Giannis Antetokounmpo (A):	8.5	11.9	5.3
Kevin Durant (D):	6.8	6.7	5.3
Joel Embiid (E):	10.1	9.8	4.2
Shai Gilgeous-Alexander (G):	9.4	4.9	5.5
LeBron James (J):	4.6	8.4	7.0
Damian Lillard (L):	7.4	3.9	7.1
Donovan Mitchell (M):	5.2	3.9	4.8
Jayson Tatum (T):	7.4	8.3	4.3

Dataset D_2 :



1. Compute the skyline (call it S_1) of D_1 with SFS. Indicate the dominance tests you made.
2. Let D_2 be as D_1 but without the *AST* attribute (see the figure). Compute, as above, the skyline (call it S_2) of D_2 .
3. Indicate the players in the 2-skyband (call it B_2) of D_2 .
4. **Bonus.** Can any of the players in $B_2 - S_2$ be a top-1 player? If so, for which (monotonic) scoring function? if not, could there be any such player in another dataset?

Solution.

1. We first sort the dataset, e.g., by *FTM* (and then by *REB* and *AST* if there are ties):

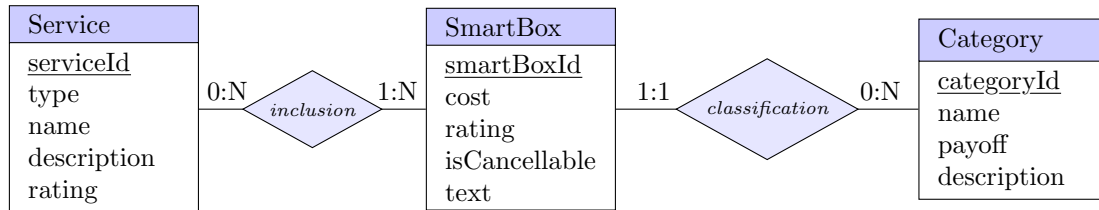
	<i>FTM</i>	<i>REB</i>	<i>AST</i>
E:	10.1	9.8	4.2
G:	9.4	4.9	5.5
A:	8.5	11.9	5.3
T:	7.4	8.3	4.3
L:	7.4	3.9	7.1
D:	6.8	6.7	5.3
M:	5.2	3.9	4.8
J:	4.6	8.4	7.0

- The skyline S_1 is $\{E, G, A, L, J\}$, which we obtain with 18 dominance tests: $E \not< G$, $E \not< A$, $G \not< A$, $E \not< T$, $G \not< T$, $A < T$, $E \not< L$, $G \not< L$, $A \not< L$, $E \not< D$, $G \not< D$, $A < D$, $E \not< M$, $G < M$, $E \not< J$, $G \not< J$, $A \not< J$, $L \not< J$. Of course the dominance tests may differ if a different criterion is used to sort the dataset.
2. We can reuse the same sorted dataset from the previous point and just drop the *AST* attribute. Here, 7 dominance tests are needed: $E < G$, $E \not< A$, $E < T$, $E < L$, $E < D$, $E < M$, $E < J$. The skyline S_2 is $\{E, A\}$.
 3. The 2-skyband B_2 contains all players in the skyline (E and A) as well as player G (dominated only by E). All the other players are dominated by more than one player.
 4. $B_2 - S_2$ only contains player G , which can never be top-1 (for any monotonic scoring function) since it is worse than E on both dimensions, so any monotonic scoring function of *FTM* and *REB* would rank E before G . In some other dataset, however, a player in $B_2 - S_2$ might end up being a tied top-1, as we might have a dominated player that has exactly the same score as the dominating player on (at least) one attribute; therefore, a monotone scoring function that only uses that attribute would rank both players equally. For instance, if G 's *FTM* score was 10.1 (same as E) instead of 9.4, G would still be dominated by E in D_2 but a function such as $f(x, y) = x$ (where x stands for *FTM* and y for *REB*) would rank both E and G as top-1 players (tied).

D. JPA (7 points)

An application of an e-commerce company manages the selling of packaged services called “smart boxes” and allows the inspection of the commercial data. The database is organized according to the conceptual schema below. The application permits an employee to select a service and check the smart boxes containing it, open one such smart box and check its details including the information of the category it belongs to (e.g., “travel”, “luxury”, “food”, “sports”, etc.). The operator can also access a category and open the list of smart boxes referring to it ordered by cost (descending), open one such smart box and check the services it contains. The services are in the order of thousands, the smart boxes are in the order of hundreds of thousands and the categories are in the order of tens.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.



Solution.

See separate pdf file.